| AGSTU | TEKNISK RAPPORT | AGSTU |
|---|---|---|
| Författare: Darwin Rengifo<br>E-postadress : | Uppgift: TEIS ECS – Embedded Computer System<br>Fil: darwin_rengifo_HWSW_task_9<br>Version : 1 | |

# HW/SW PROTOTYP AV ONCHIP FLASH SYSTEM

**Darwin Rengifo**
**2020-07-29**

Sammanfattning:
Denna rapport beskriver en förstudie och planering för utvecka kunskap hur ett Onchip Flash läggs till för IntelFPGA DE10-Lite kortet, systemet använder en bytrare SW[9] för visar ”bootningen” system. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1. Inledning ..... 2
2. Kravspecifikation från kund ..... 3
3. Projekplan ..... 4
4. Beskrivning av hw/sw-konstruktionen ..... 5
4.1 Skapa ett Qsys projekt ..... 5
4.2 Skapa top level fil, och pin assignments ..... 11
4.3 Quartus inställningar ..... 13
4.4 Skapa mjukvara och hexadecimalt fil (.hex) ..... 15
4.5 Skapa .pof fil och programmera kortet ..... 17
5. Hur ”bootningen” fungerar ..... 20
6. Validering ..... 20

# 1. Inledning

Denna rapport beskriver HW/SW-konstruktionen för ett internt flashminne som ligger i kortet DE10-Lite och hur för ”bootningen” fungerar i ett inbyggt system.

# 2. Kravspecifikation från kund

*Tabell 1. Kravspecifikation från kund*

| Krav | Beskrivning | Utfört |
|---|---|---|
| | ***Krav*** | Ja/nej |
| 001 | Skriv ett program som bootas upp DE10-Lite kortet och räknar från 0 till 9 på en 7-segments display. | Ja |
| 002 | Skriv en kort beskrivning av HW/SW-konstruktionen och hur ”bootningen” fungerar. | Ja |
| 003 | Validering ska vara en kort film (max 10 sekunder) som visar när strömmen slås på och att 7-segments displayen räknar upp. | Ja |
| | ***Leverans*** | |
| 004 | Leveransen ska ske till plattformen Itslearning. Leveransen ska vara beskrivningen (krav 002) och en länk till filmen. Rapporten ska också innehålla kostnaden för konsultuppdraget. Företaget fakturerar 600 SEK/timme för projektet. Namnet på filen ska vara ”fornamn_efternamn_HWSW_task_9”. Sista leveransdag se Itslearning (för VG). | Ja |

# 3. Projekplan

Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 18 timmar/600kr(timme), vilket ger en lönekostnad på 10 800 SEK.

***Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet***

| **Aktivitet** | | | |
|---|---|---|---|
| | **31** | **Timmar(h)** | **Kostnad(kr)** |
| Kravspec./förstudie | 5 | 5 | 3 000 |
| Arkitekturbeskrivning | 4 | 4 | 2 400 |
| Programmering av HW/SW gränssnitt | 6 | 6 | 3 600 |
| Validering | 1 | 1 | 6 00 |
| Slutrapport | 2 | 2 | 1 200 |
| Summa tid | 18 | 18 | 10 800 |

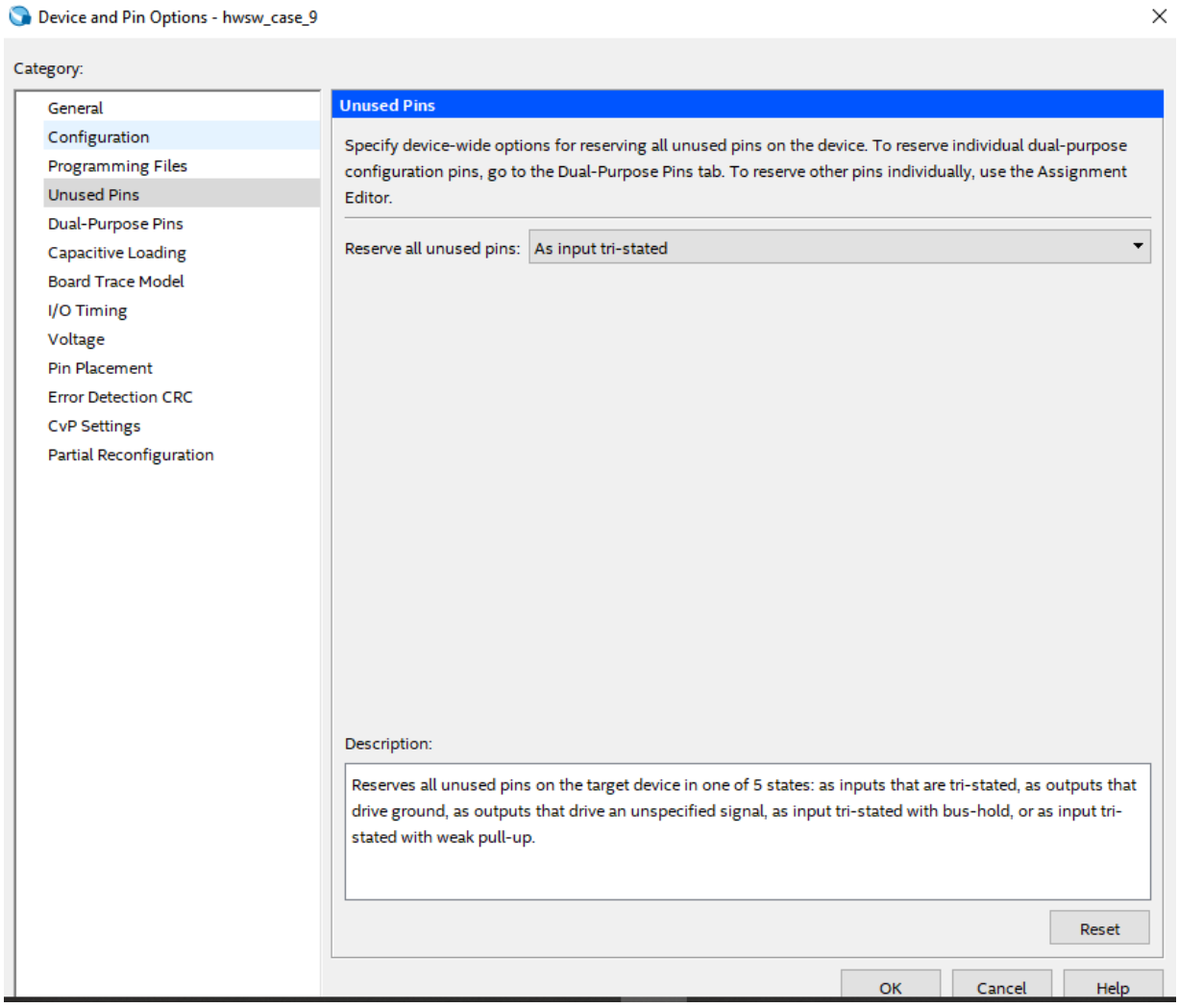
# 4. Beskrivning av hw/sw-konstruktionen

FIGUR 1 visar det konfigureringen för att bygga ”bootningen” systemet med användningen av internt flashminne.

## 4.1 Skapa ett Qsys projekt

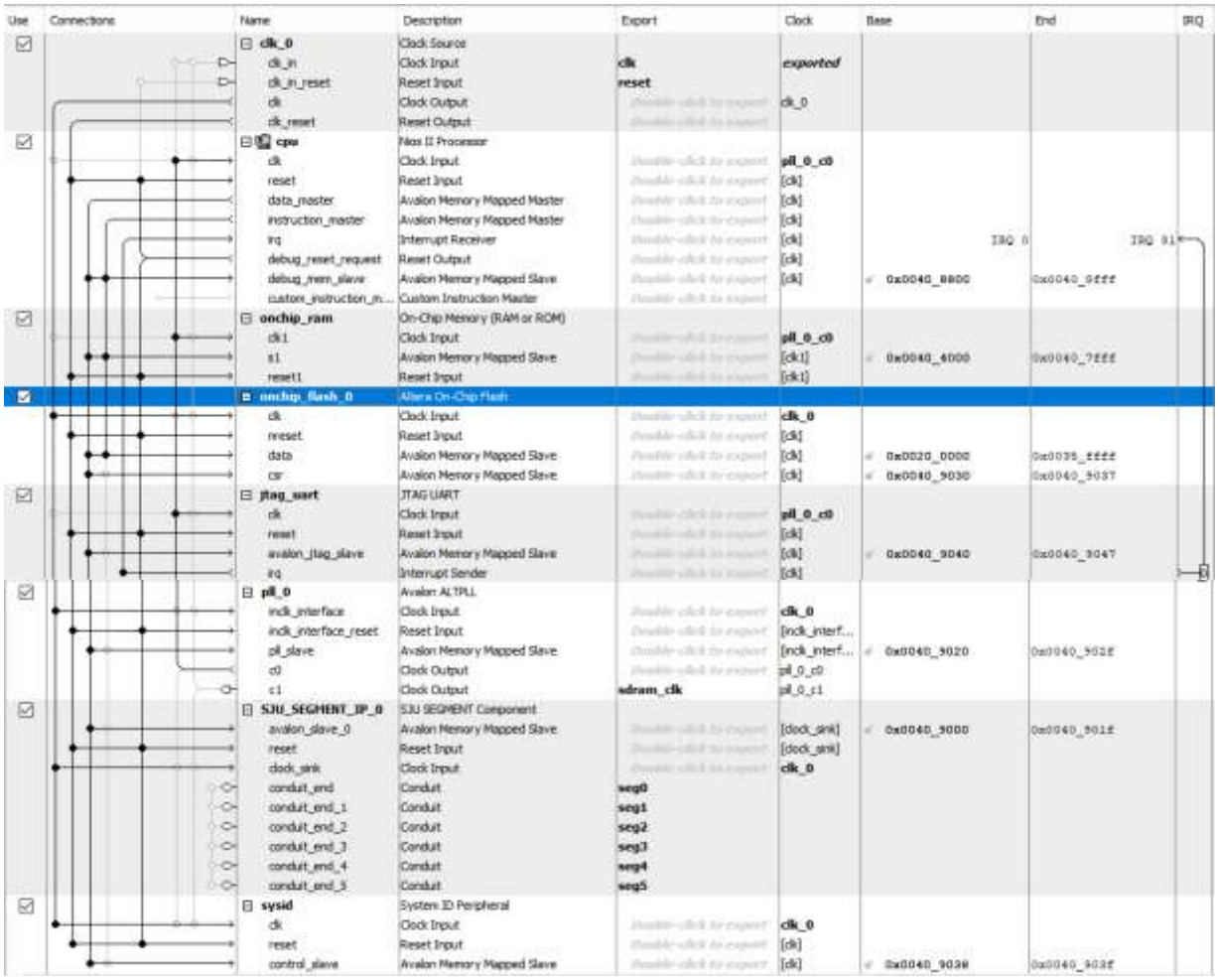
Börja med att skapa ett nytt projekt i Quartus. I exemplet kommer detta projekt att heta ”hwsw_case_9”.

Projektet kommer inte att använda alla pins på FPGA. Reserverar alla oanvända pins på målenheten i en "Tri-State" tillstånd. Se figur 1.



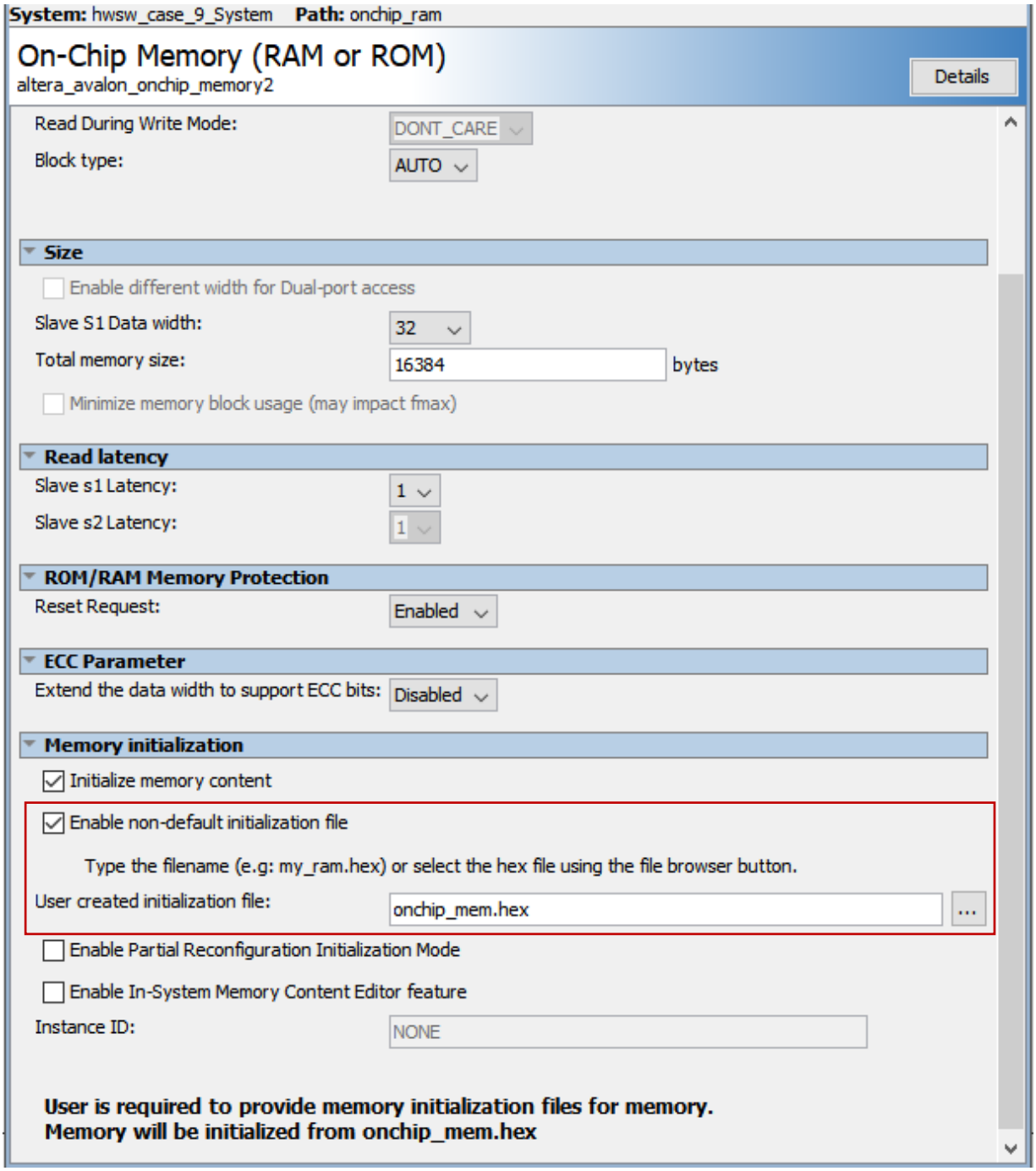

**Figur 1 Välj Unused pins**

Starta Qsys och lägg till de komponenter som behövs för att kan jobba med projektet. Det finns en IP komponente ”SJU_SEGMEMENT_IP_0” som har skapat för att kan jobbar logiken av 7-segments display på hårdvaran. Se figur 2.



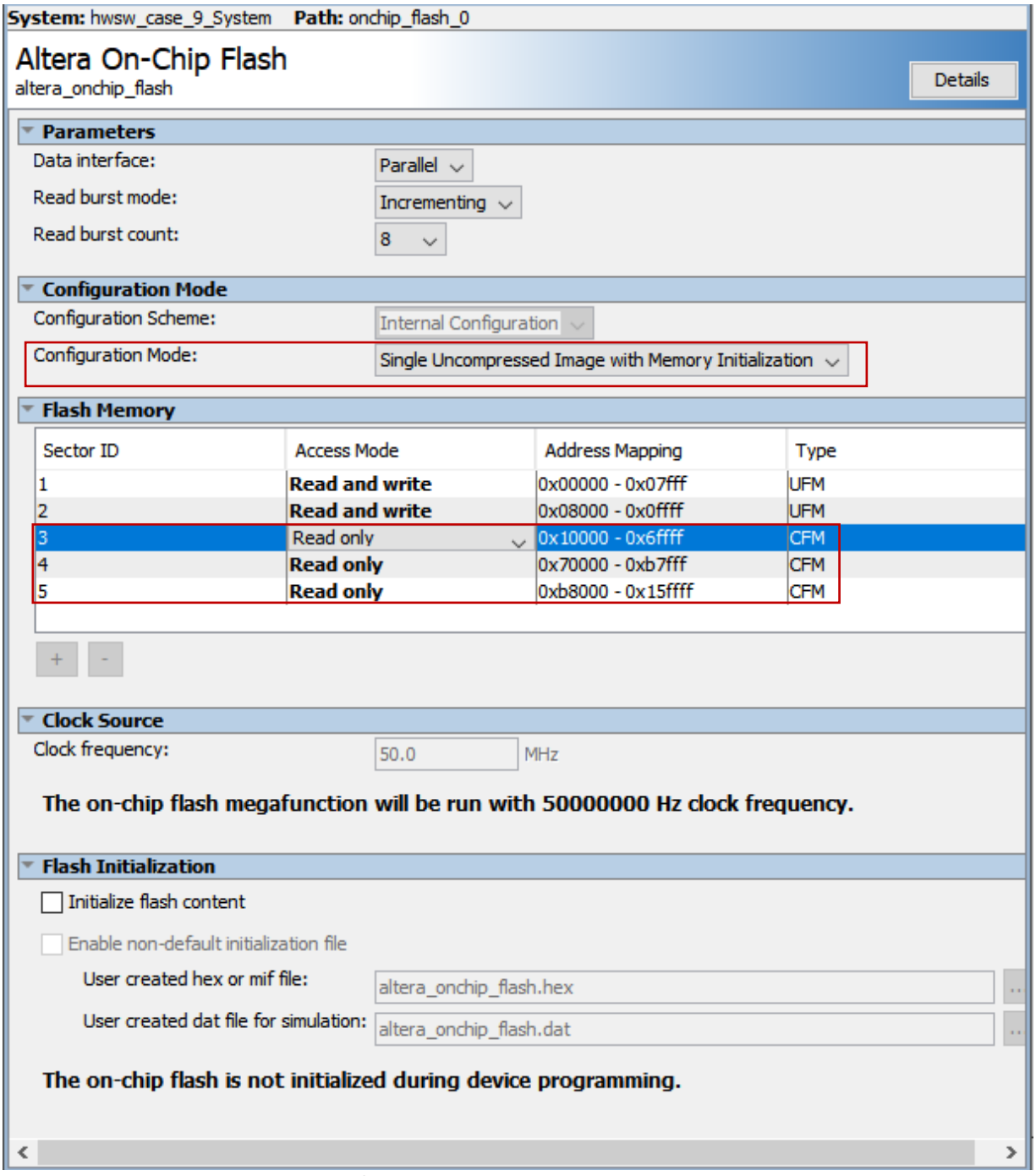

**Figur 2 Välj komponent onchip_flash_0**

Nästa figur visar hur onchip memory komponenten ska vara konfigurerad för att användas tillsammans med onchip flash minne. Aktiverar "enable non-defaul initializacion file" i delen av "memory initialization".
På "User created initialization file" det kommer att skapat automatisk ett namn som heter"onchip_mem.hex" , det kommer att använda sen det namnet för att skapar hexadecimalt filen som kommer att innehåla mjukvaran, ska vi se den här prodcedur senare. Se figur 3.



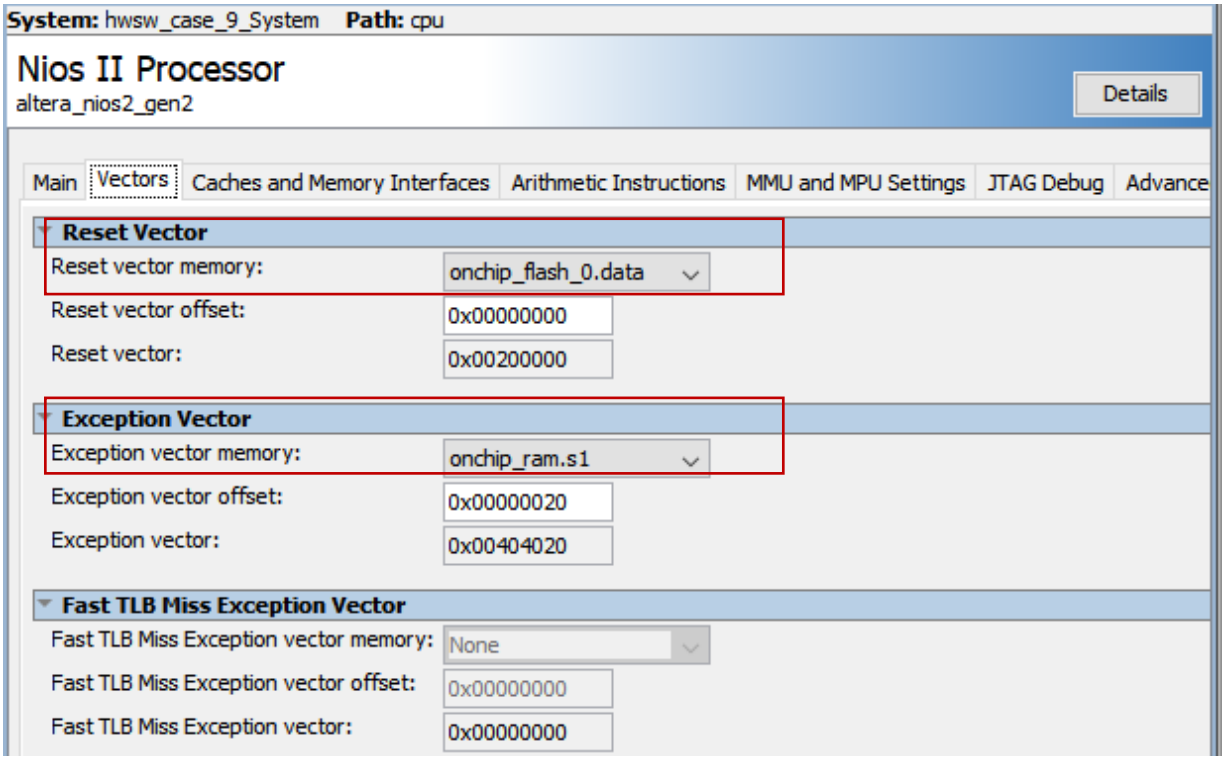

**Figur 3 Konfiguration av komponenten On-Chip Memory**

Nästa figur visar hur onchip flash komponenten ska vara konfigurerad för att användas tillsammans med ett internt RAM minne. Ändra Configuration Mode till "Single Uncompressed Image with Memory Initialization", och ändra Sector ID 3,4,5 till "Read only". I delen av "Flas Memory" det observerar sig på "Type" kolumn att det interna flashminnet är uppdelat i två delar som är CFM (Configuration Flash Memory) var hårdvarukonfiguration (.sof) sparas och UFM (User Flash Memory) var mjukvara sparas (.hex). Se figur 4.



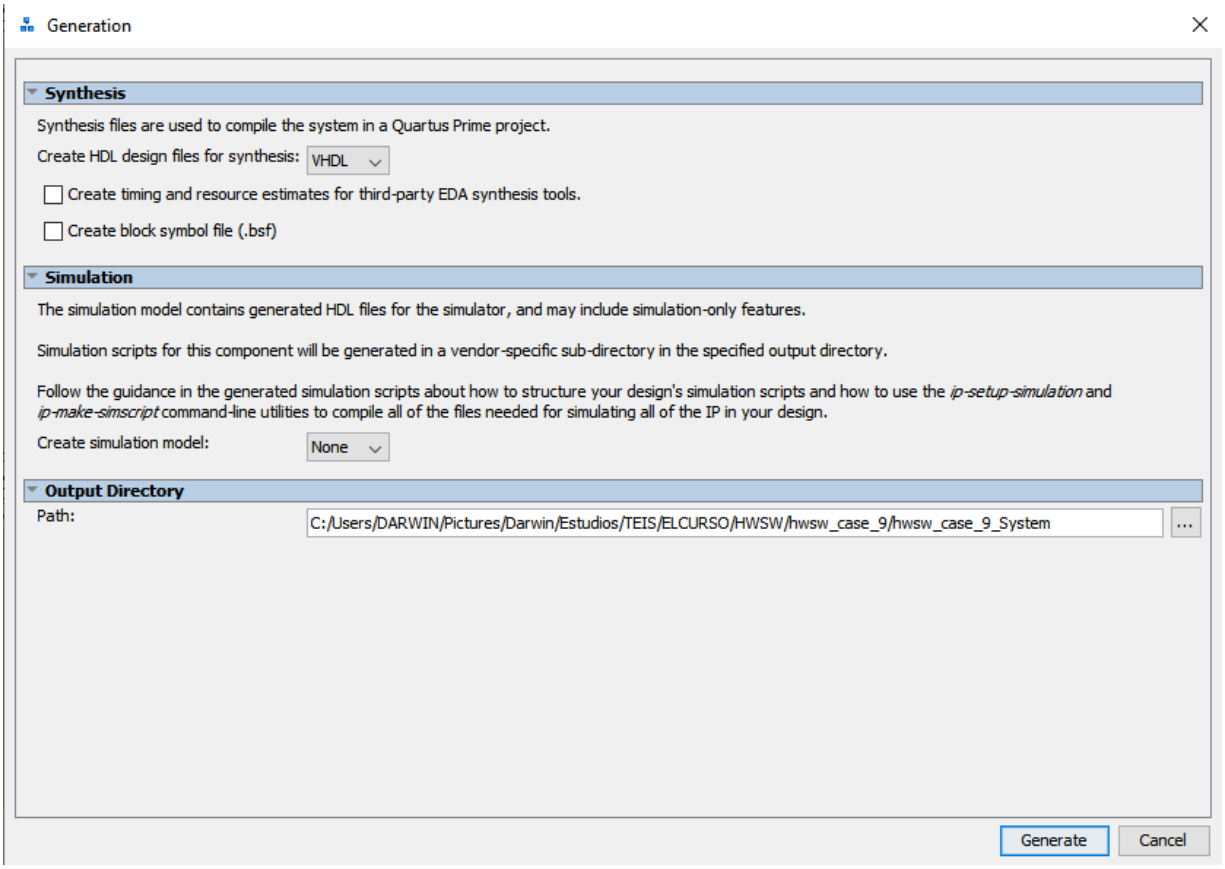

**Figur 4 Konfiguration av komponenten On-Chip Flash Memory**

Nästa steg är konfigurerar parameter för Nios ll CPU, clicka på "Vectors". På "Reset Vector" delen ställ in ”Reset vector memory” till ”onchip_flash_0.data” eftersom  systemet ska starta upp från flashminnet efter en reset SW[9].
På ”Exception Vector" delen ställ in ”Exception vector memory” till ”onchip_ram.s1” för att exekverar koden som finns i detta minne efter att koden var kopieras från flashminnet till detta minnet . Se nästa figur 5.




**Figur 5 Konfiguration av komponenten Nios II Processor**

För att integrerar systemet i Quartus ställ in ”VHDL” på ”Create HDL design files for synthesis” delen. Se figur  6.




**Figur 6 Genererar systemet i Quartus II**

## 4.2 Skapa top level fil, och pin assignments

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity hwsw_case_9 is
   port (
      CLOCK_50    : in    std_logic                       := '0';
      SW          : in    std_logic                       := '0';
      HEX0        : out   std_logic_vector(6 downto 0);
      HEX1        : out   std_logic_vector(6 downto 0);
      HEX2        : out   std_logic_vector(6 downto 0);
      HEX3        : out   std_logic_vector(6 downto 0);
      HEX4        : out   std_logic_vector(6 downto 0);
      HEX5        : out   std_logic_vector(6 downto 0)
   );
end entity hwsw_case_9;

architecture rtl of hwsw_case_9 is


   component hwsw_case_9_System is
      port (
      clk_clk       : in    std_logic                       := '0';
      reset_reset_n : in    std_logic                       := '0';
      sdram_clk_clk : out   std_logic;
      seg0_export   : out   std_logic_vector(6 downto 0);
      seg1_export   : out   std_logic_vector(6 downto 0);
      seg2_export   : out   std_logic_vector(6 downto 0);
      seg3_export   : out   std_logic_vector(6 downto 0);
      seg4_export   : out   std_logic_vector(6 downto 0);
      seg5_export   : out   std_logic_vector(6 downto 0)
   );
   end component hwsw_case_9_System;

   signal tmp_sdram_clk       : std_logic;

begin

  ----qsys component

  b0v_inst_hwsw_case_9_System : component hwsw_case_9_System
     port map (
        clk_clk       => CLOCK_50,
        reset_reset_n => SW,
        sdram_clk_clk => tmp_sdram_clk,
        seg0_export   => HEX0,
        seg1_export   => HEX1,
        seg2_export   => HEX2,
        seg3_export   => HEX3,
        seg4_export   => HEX4,
        seg5_export   => HEX5
     );

end rtl;
```

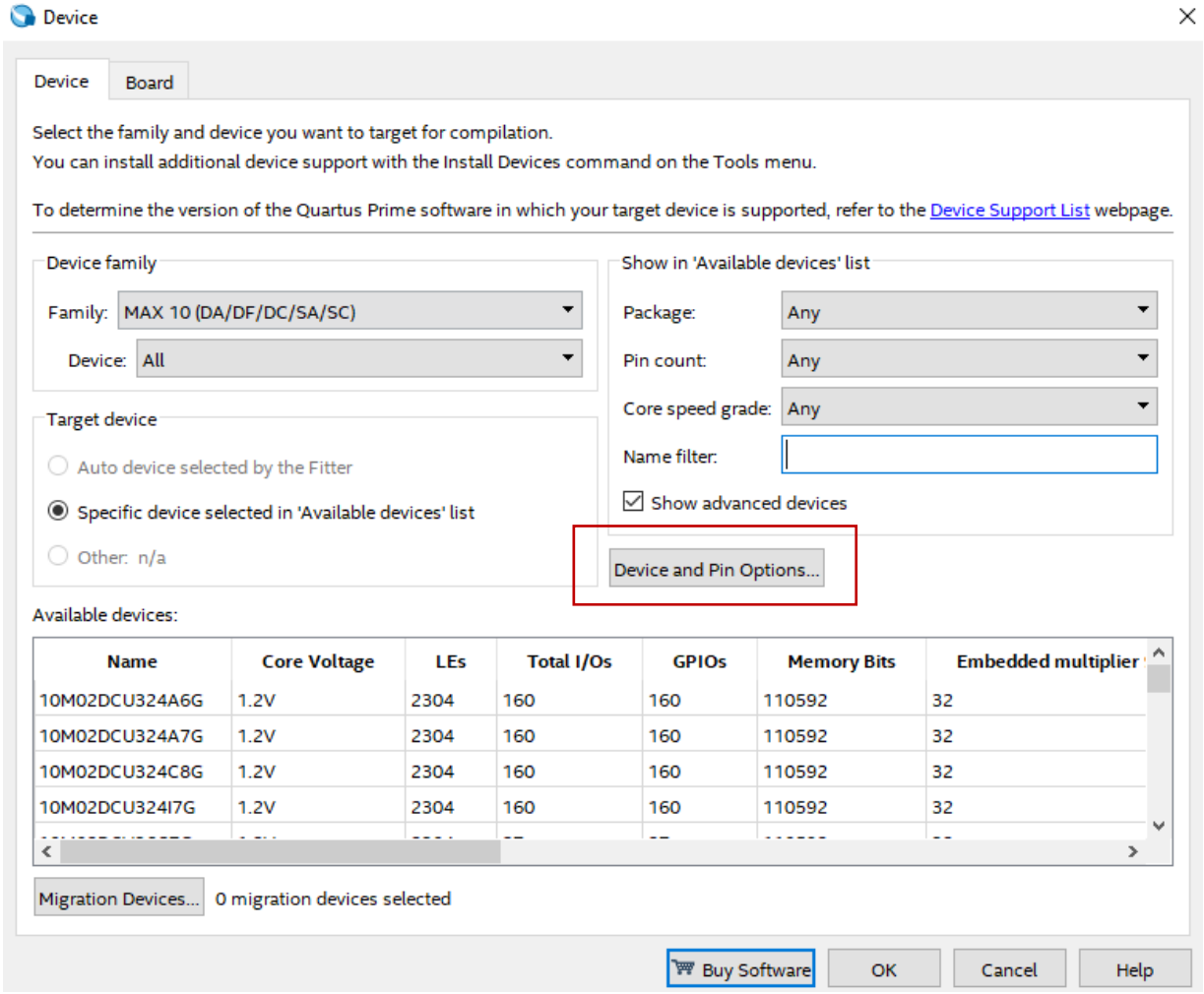
**Figur 7 Top Level system**

| Node Name | Direction | Location |
|---|---|---|
| CLOCK_50 | Input | PIN_N14 |
| HEX0[6] | Output | PIN_C17 |
| HEX0[5] | Output | PIN_D17 |
| HEX0[4] | Output | PIN_E16 |
| HEX0[3] | Output | PIN_C16 |
| HEX0[2] | Output | PIN_C15 |
| HEX0[1] | Output | PIN_E15 |
| HEX0[0] | Output | PIN_C14 |
| HEX1[6] | Output | PIN_B17 |
| HEX1[5] | Output | PIN_A18 |
| HEX1[4] | Output | PIN_A17 |
| HEX1[3] | Output | PIN_B16 |
| HEX1[2] | Output | PIN_E18 |
| HEX1[1] | Output | PIN_D18 |
| HEX1[0] | Output | PIN_C18 |
| HEX2[6] | Output | PIN_B22 |
| HEX2[5] | Output | PIN_C22 |
| HEX2[4] | Output | PIN_B21 |
| HEX2[3] | Output | PIN_A21 |
| HEX2[2] | Output | PIN_B19 |
| HEX2[1] | Output | PIN_A20 |
| HEX2[0] | Output | PIN_B20 |
| HEX3[6] | Output | PIN_E17 |
| HEX3[5] | Output | PIN_D19 |
| HEX3[4] | Output | PIN_C20 |
| HEX3[3] | Output | PIN_C19 |
| HEX3[2] | Output | PIN_E21 |
| HEX3[1] | Output | PIN_E22 |
| HEX3[0] | Output | PIN_F21 |
| HEX4[6] | Output | PIN_F20 |
| HEX5[5] | Output | PIN_N19 |
| HEX5[4] | Output | PIN_M20 |
| HEX5[3] | Output | PIN_N18 |
| HEX5[2] | Output | PIN_L18 |
| HEX5[1] | Output | PIN_K20 |
| HEX5[0] | Output | PIN_J20 |
| SW | Input | PIN_F15 |

**Figur 8 Pin assignments**

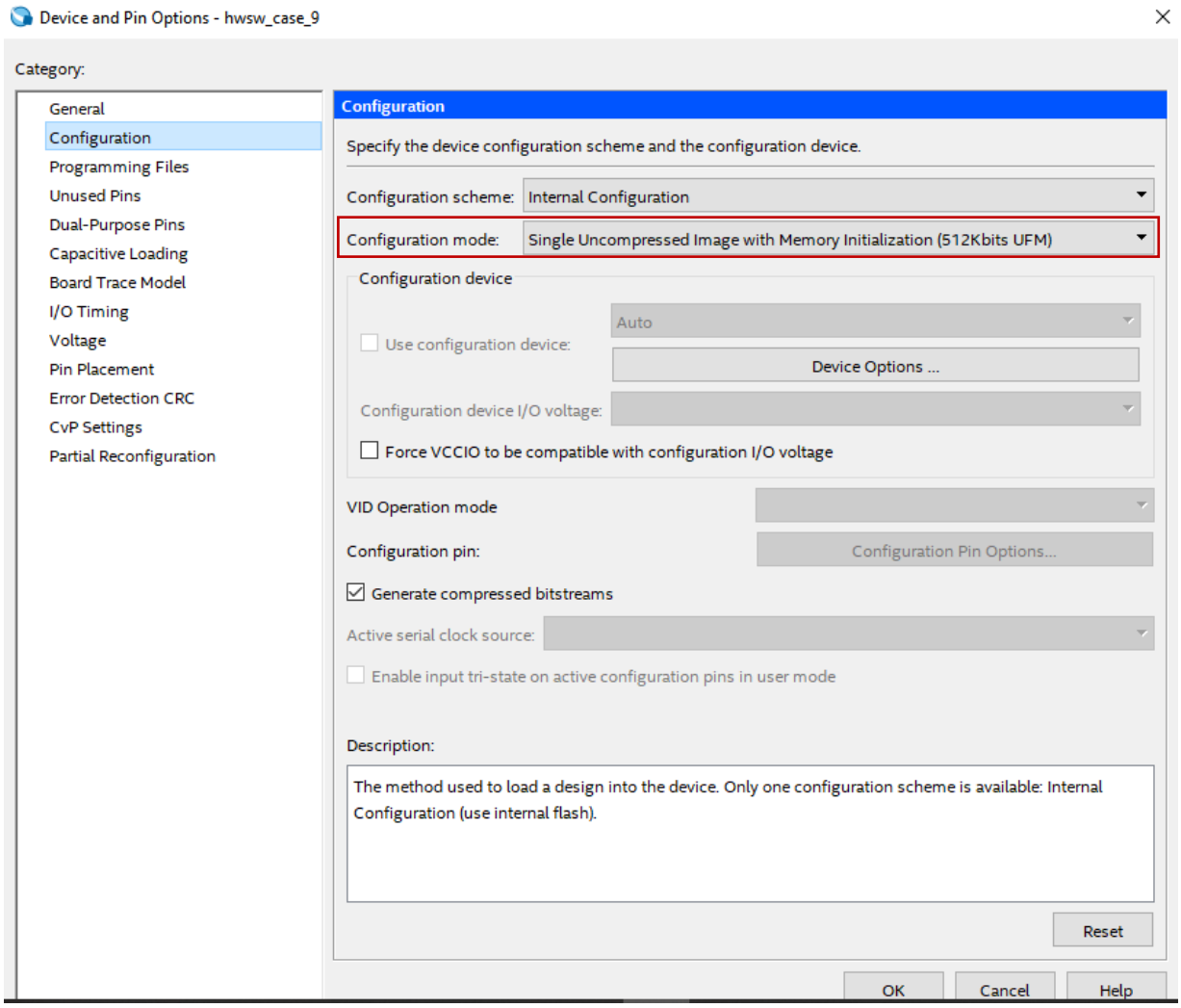
### 4.3 Quartus inställningar

Så att det inte finns något felmeddelande för minnets initiering under kompileringen måste ändras detta alternativ på nästa steg. Klicka på nästa alternativ ”Assignment/Device…/Device and Pin Opstios…”. Se figur 9.




**Figur 9 Device configuration(1)**

På denna del det kommer att ändras till ”Single Uncompressed Image with Memory Initialization (512Kbits UFM)”. Se figur 10




**Figur 10 Device Konfiguration(2)**

Kompilera programmet.

### 4.4 Skapa mjukvara och hexadecimalt fil (.hex)

För att skapa ett nytt projekt på Nios II det kommer att använda .sopcinfo filen som skapades när systemet genererades i Qsys.
Det finns en IP komponent för att visar resultat på 7-segments display som kommer att hanterar från hårdvaran och genom mujkvaran det kommer att skriva en räknaren på register av denna komponent genom av Avalon buss interface. Denna projektet använd för kompileringen: Optimization Level=Size, Reduced device drivers, Small C library. Se figur 11.

```
#include <alt_types.h>
#include <system.h>
#include <stdio.h>
#include <sju_segment.h>

int main()
{
    int data=0, i=0, xfin=0;
    while (1)
    {
        //skriver data via avalon buss interface
        SJU_SEGMENT_WRITE(base,data);
        data++;
        if(data > 100){data = 0;}
        for(i=0; i<500000; i++);
      }

return 0;
}
```

**Figur 11 C koden för 7-segments display.**

Det behöver onchip_ram adressen för att skapa .hex fil.se figur 12.

## Nios II Memory Map

| Slave Descriptor | Address Range | Size | Attributes |
|---|---|---|---|
| jtag_uart | 0x00409040 - 0x00409047 | 8 | printable |
| sysid | 0x00409038 - 0x0040903F | 8 | |
| onchip_flash_0_csr | 0x00409030 - 0x00409037 | 8 | |
| pll_0 | 0x00409020 - 0x0040902F | 16 | |
| SJU_SEGMENT_IP_0 | 0x00409000 - 0x0040901F | 32 | |
| onchip_ram | 0x00404000 - 0x00407FFF | 16384 | memory |
| onchip_flash_0_data | 0x00200000 - 0x0035FFFF | 1441792 | flash, memory, non-volatile |

**Figur 12 Memory-map från "sumary.html"**

Nästa figur visa instruktionen "elf2hex" för att skapa hexadecimalt fil för "bootloadern". På denna del mjukvara sparas på en hexadecimalt fil (.hex). Se figur 13.




**Figur 13 Hexadecimalt fil till bootloadern**

## 4.5 Skapa .pof fil och programmera kortet

Efter att skapa det fil "onchip_mem.hex" sparades den i projektmapp som skapade på Nios II. Filen "onchip_mem.hex" måste flytta till innuti Quartus II projektmapp för att kan inkluderar det i projektet under kompilering. Se figur 14.

| | | | |
|---|---|---|---|
| .qsys_edit | 2020-07-27 19:55 | Filmapp | |
| db | 2020-07-27 19:50 | Filmapp | |
| hdl | 2020-07-27 18:43 | Filmapp | |
| hwsw_case_9_System | 2020-07-27 19:56 | Filmapp | |
| incremental_db | 2020-07-27 20:54 | Filmapp | |
| IP | 2020-07-27 18:44 | Filmapp | |
| output_files | 2020-07-27 20:55 | Filmapp | |
| simulation | 2020-07-27 20:58 | Filmapp | |
| hwsw_case_9 | 2020-07-27 19:50 | QPF-fil | 2 kB |
| hwsw_case_9.qsf | 2020-07-27 19:50 | QSF-fil | 14 kB |
| hwsw_case_9.qsf.bak | 2020-07-27 21:01 | BAK-fil | 4 kB |
| hwsw_case_9_System | 2020-07-27 19:56 | QSYS-fil | 60 kB |
| hwsw_case_9_Syste... | 2020-07-27 20:30 | SOPCINFO-fil | 330 kB |
| onchip_mem.hex | 2020-07-29 00:00 | HEX-fil | 39 kB |

**Figur 14 Hexadecimalt fil till bootloadern**

Efter att följer alla föregående stegen, projektet har kompilerat och visar den följande resultat. Se figur 15.

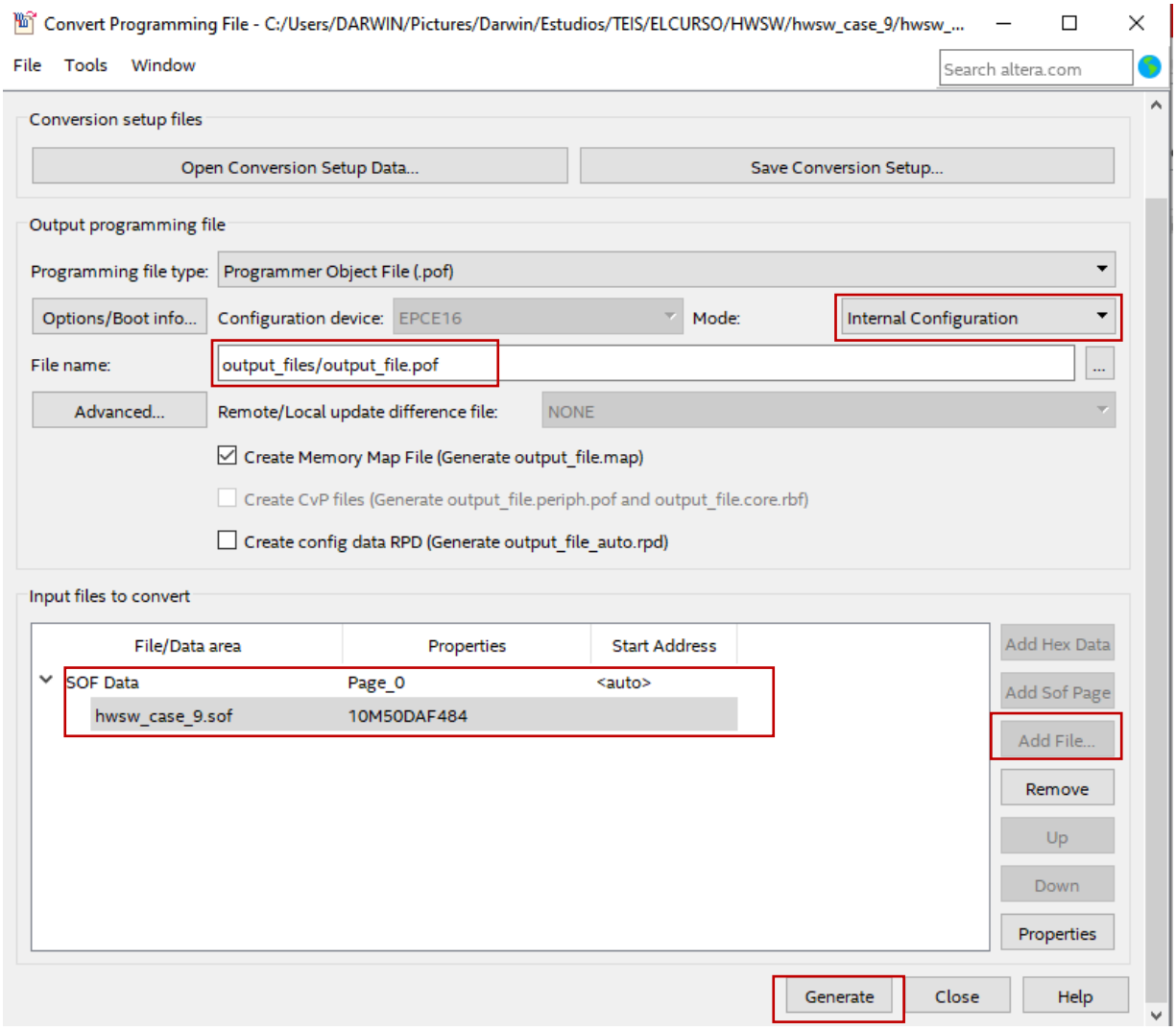
**Flow Summary**

<<Filter>>

| | |
|---|---|
| Flow Status | Successful - Wed Jul 29 00:18:01 2020 |
| Quartus Prime Version | 17.0.0 Build 595 04/25/2017 SJ Lite Edition |
| Revision Name | hwsw_case_9 |
| Top-level Entity Name | hwsw_case_9 |
| Family | MAX 10 |
| Device | 10M50DAF484C7G |
| Timing Models | Final |
| Total logic elements | 3,323 / 49,760 ( 7 % ) |
| Total combinational functions | 2,652 / 49,760 ( 5 % ) |
| Dedicated logic registers | 2,225 / 49,760 ( 4 % ) |
| Total registers | 2225 |
| Total pins | 44 / 360 ( 12 % ) |
| Total virtual pins | 0 |
| Total memory bits | 142,336 / 1,677,312 ( 8 % ) |
| Embedded Multiplier 9-bit elements | 0 / 288 ( 0 % ) |
| Total PLLs | 1 / 4 ( 25 % ) |
| UFM blocks | 1 / 1 ( 100 % ) |

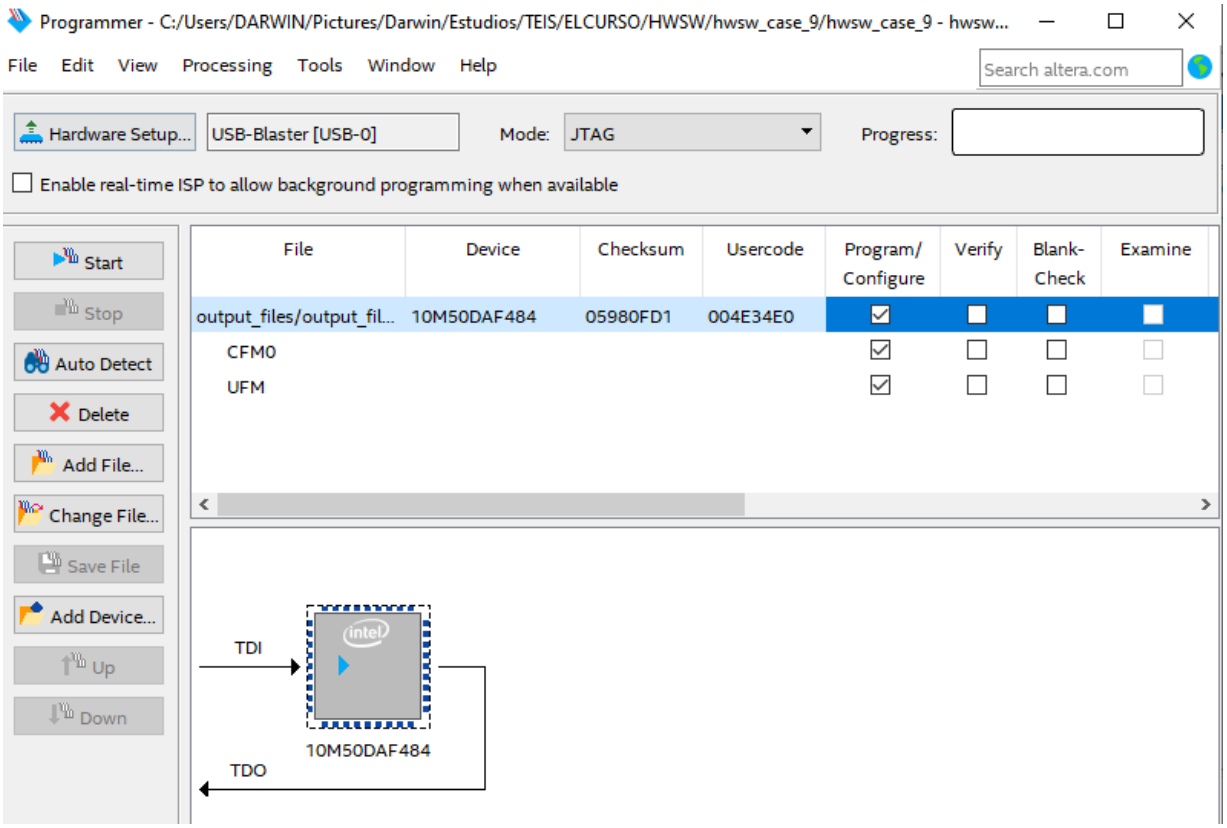
**Figur 15 Resultaten efter kompileringen**

Nu ska binärfilen med mjukvaran, och hårdvaran läggas ihop till en .pof fil. Öppnas menyn ”File/Convert Programming Files…” för att körs nästa steg. Det ändras ”Internal Configuration”, markera SOF Data och välj ”Add File…” , leta reda på hwsw_case_9.sof filen som skapades när projektet kompilerades i Quartus. Som sista del tryck på knappen Generate. Efter att göra den steg det kommer att skapas en fil med namn ”output_file.pof”. Se figur.16.




**Figur 16 Skapas .pof fil**

För att ladda upp .pof fil till DE10-Lite kortet öppnas menyn ”Tools/Programmer" och välja ”output_file.pof” från korrekt mappen. Se figur 17.



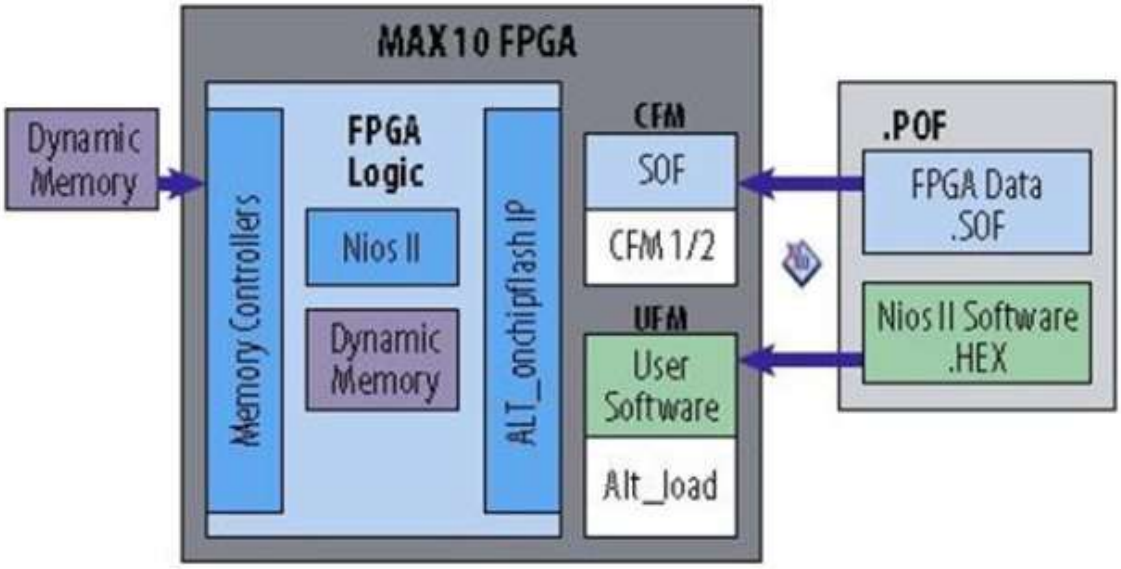

**Figur 17 Programmer**

# 5. Hur ”bootningen” fungerar

DE10-Lite kortet innehålla en ett internt flashminne som kan sparas information även när kortet får inte ingen spänning. Det interna flashminne är uppdelat i två delar som är CFM (Configuration Flash Memory) var hårdvarukonfiguration (.sof) sparas och UFM (User Flash Memory) var mjukvara sparas (.hex).

Vid uppstart kan både hårdvara och mjukvara laddas in i FPGA:n. Först kommer hårdvaran att laddas upp, och sedan används Nios CPU:n att flytta mjukvaran från UFM till det minne som ska användas för att exekvera programkoden. Denna kopieringsprocess kallas för bootloader.

Figuren nedan visar ett blockschema på en MAX10 krets. På höger sida av bilden kan det utläsas att hårdvaran, och mjukvaran läggs i en fil som heter .pof. Denna fil kommer sedan att bli uppdelad i två olika minnen inuti FPGA:n.
Det går att välja om Niosll CPU:n ska skicka programmet till ett internt minne (Dynamic Memory inuti kretsen), eller ett externt SDRAM minne (Dynamic Memory utanför kretsen).




**Figur 1: EERAM inuti MAX 10 FPGA kretsen**

# 6. Validering

https://www.youtube.com/watch?v=RuG1jVtIkiE